

**PROJECT REPORT
ON**

**Customer Sentiment Analysis & Review Mining for E-Commerce
Apparel**

Project Report submitted to GITAM (Deemed to be University) as a partial fulfillment of the
requirements for the award of the Master of Business Administration

Submitted By:

Kumudini — 2025337529

Shreya — 2025027663

Neeraj — 2025008753

Meghana — 2025724130

Likhitha — 2025679727



GITAM
DEEMED TO BE UNIVERSITY

GITAM SCHOOL OF BUSINESS
GITAM (DEEMED TO BE UNIVERSITY)
HYDERABAD - 502329, INDIA

**A
Business Text Analytics Project Report
ON**

Customer Sentiment Analysis & Review Mining for E-Commerce Apparel

A Project Report submitted to GITAM School of Business in partial fulfilment
of the requirement for the award of the degree of
MASTER OF BUSINESS ADMINISTRATION (SESSION: 2025-27) - MBA 5th TERM

Submitted by:

Kumudini — 2025337529

Shreya — 2025027663

Neeraj — 2025008753

Meghana — 2025724130

Likhitha — 2025679727



GITAM
(DEEMED TO BE UNIVERSITY)
Rudraram, Patancheru (M), Sangareddy Dist., Telangana

1. Business Problem Definition

For any online clothing store, customer reviews are one of the most useful sources of direct feedback. But when an e-commerce platform sells hundreds of products and receives thousands of reviews, it is basically impossible for human teams to read through each comment one by one.

While shoppers leave quick star ratings between 1 and 5, numeric ratings don't actually explain what went wrong with a product. A customer might drop a 2-star rating because the fabric felt paper-thin, while another person gave 2 stars simply because the sleeves were too long. In fashion retail, return rates often shoot past 25%, causing huge reverse shipping costs and lost profits.

The main purpose of our project is to build an automated text analytics pipeline using Python and NLP. By classifying reviews into positive and negative sentiments and finding the exact words customers use repeatedly, the business can quickly fix sizing charts, audit fabric quality, and reduce avoidable product returns.

2. Literature Support & Background

Text analytics and sentiment mining have become standard tools in e-commerce for tracking customer feedback and brand health. While early methods relied mostly on static sentiment dictionaries (like VADER) or rule-based word matching, these traditional techniques often struggle with industry-specific terms, fashion phrasing, or sarcastic comments.

In business applications, pairing TF-IDF (Term Frequency-Inverse Document Frequency) feature extraction with simple machine learning models like Logistic Regression remains one of the most reliable and widely used baselines. It runs fast, uses very little computing power, and lets us look directly at feature coefficients so business teams can see which exact words influence customer ratings the most.

3. Data Collection & Description

We worked with the public *Women's E-Commerce Clothing Reviews* dataset collected from Kaggle. The dataset contains actual customer reviews and purchasing data from an online womens apparel store.

- **Total Records:** 23,486 rows and 10 columns.

Key Columns Used:

- **Review Text:** The written feedback left by customers.
- **Rating:** Customer rating from 1 to 5 stars.
- **Department Name:** Product department like Tops, Dresses, Bottoms, etc.
- **Recommended IND:** Binary flag (0 or 1) indicating if the customer recommends the item.

Target Label Formulation: To create clear classes for our supervised model, we converted the ratings into binary categories:

- **Positive (1):** Ratings 4 and 5.
- **Negative (0):** Ratings 1 and 2.
- **Neutral Dropped:** Reviews with a 3-star rating were removed because they contain mixed opinions that often confuse a binary classifier.

Handling Missing Data: All blank or null entries in the review text column were dropped before starting the text pipeline.

```
import pandas as pd

file_path = "/kaggle/input/datasets/nicapotato/womens-ecommerce-clothing-reviews/Womens Clothing E-Commerce Reviews.csv"
df = pd.read_csv(file_path)
df.head()
```

	Unnamed: 0	Clothing ID	Age	Title	Review Text	Rating	Recommended IND	Positive Feedback Count	Division Name	Department Name	Class Name
0	0	767	33	NaN	Absolutely wonderful - silky and sexy and comf...	4	1	0	Intimates	Intimate	Intimates
1	1	1080	34	NaN	Love this dress! it's sooo pretty. i happene...	5	1	4	General	Dresses	Dresses
2	2	1077	60	Some major design flaws	I had such high hopes for this dress and reall...	3	0	0	General	Dresses	Dresses
3	3	1049	50	My favorite buy!	I love, love, love this jumpsuit. it's fun, fl...	5	1	0	General Petite	Bottoms	Pants
4	4	847	47	Flattering shirt	This shirt is very flattering to all due to th...	5	1	6	General	Tops	Blouses

(Figure 1 — Initial table preview of the *Women's E-Commerce Clothing Reviews* dataset.)

4. Data Preprocessing & System Architecture

Raw review text is full of slang, punctuation, and capital letters, so we had to clean it thoroughly before feeding it into any model. We handled the preprocessing using Python's `re` library and `nltk`.

4.1 Preprocessing Pipeline

1. **Lowercasing:** Converted all review text to lowercase so words like "Dress" and "dress" are treated identically.
2. **Punctuation & Special Character Removal:** Used regular expressions to strip out punctuation marks, digits, and special symbols, leaving behind only clean words.
3. **Stopword Removal:** Removed common filler words such as "the", "and", "is", and "at" using NLTK's English stopwords list.
4. **Lemmatisation:** Reduced words to their base dictionary roots using NLTK's WordNetLemmatizer (for instance, converting "flattering" to "flatter" and "dresses" to "dress").

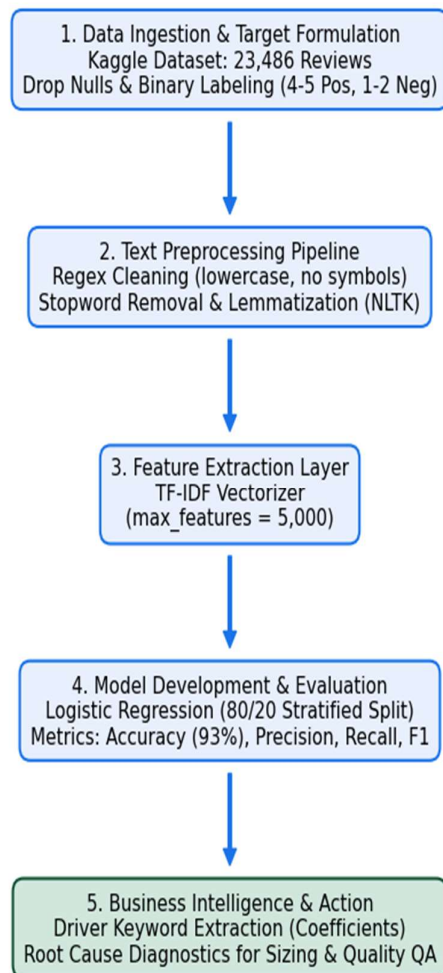
4.2 Feature Extraction (TF-IDF Vectorisation)

Machine learning models cannot read raw strings, so we transformed the cleaned text into numerical vectors using `TfidfVectorizer` from `scikit-learn`. We set `max_features=5000` to capture the most meaningful words while keeping the dataset lightweight and filtering out rare typos.

4.3 End-to-End System Architecture

Our project follows a structured five-step workflow: ingesting the dataset, cleaning the text, vectorizing words with TF-IDF, training our classifier, and extracting the top business keywords.

End-to-End NLP Project Architecture



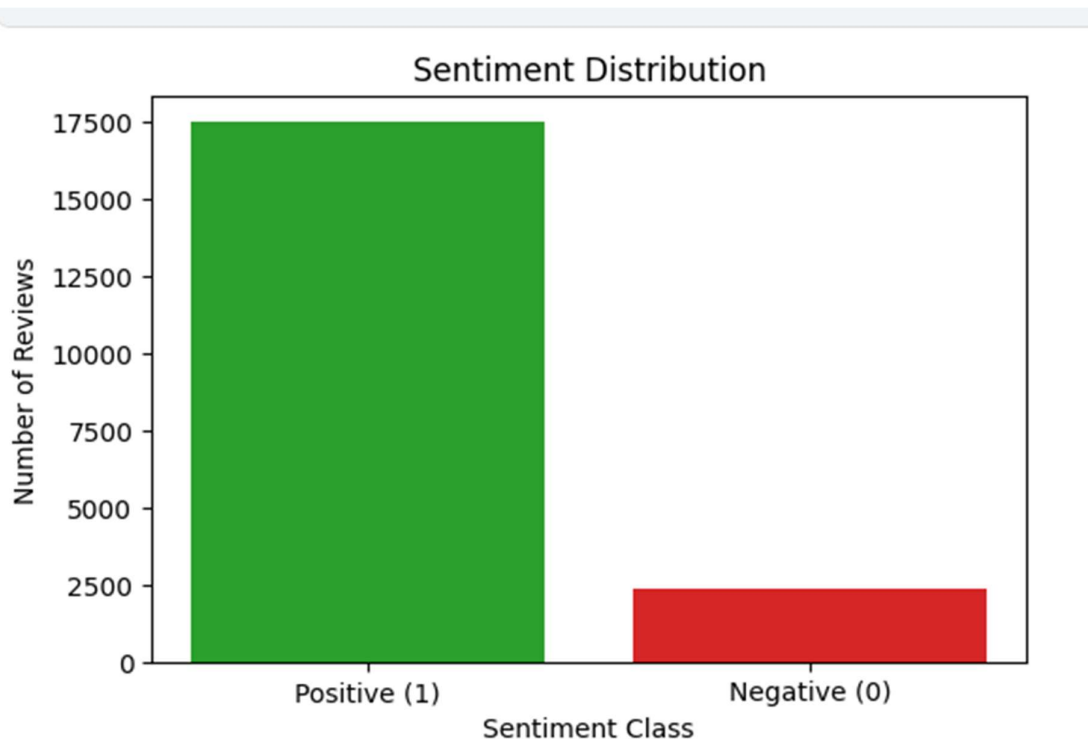
(Figure 2 — Technical workflow of our NLP sentiment analysis pipeline.)

5. Exploratory Text Analysis & Key Visualisations

Before running the models, we explored the text data to understand word frequencies and class distribution across categories.

5.1 Class Distribution

The dataset has a significant class imbalance: satisfied customers left far more positive reviews than negative ones. This is very common in online shopping because buyers usually post reviews when they really love an item or when something goes horribly wrong.



(Figure 3 — Bar chart showing the split between positive and negative reviews.)

5.2 WordCloud Analysis

We plotted side-by-side WordClouds for positive and negative reviews to spot recurring patterns visually:

- **Positive Reviews:** Words like *love*, *comfortable*, *great*, *perfect*, *soft*, and *flattering* appeared most frequently. This shows that fabric feel and good fitting are the main reasons customers leave good feedback.
- **Negative Reviews:** Words like *disappointed*, *cheap*, *unflattering*, *huge*, *poor*, and *return* dominated the feedback. This highlights that loose fits and poor material quality are the main culprits behind bad reviews.

6. Model Development & Evaluation

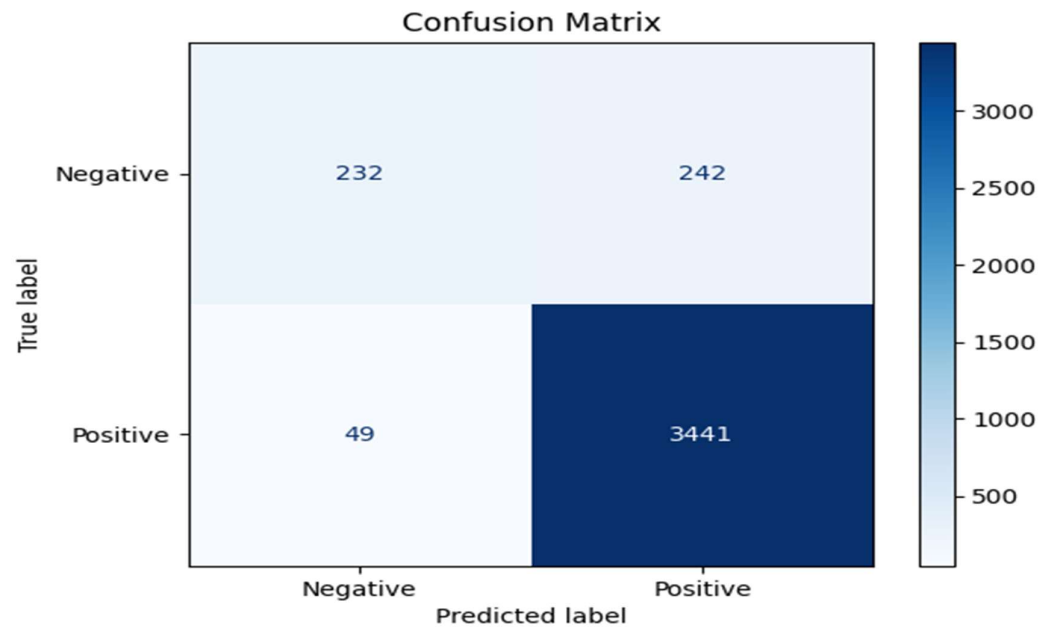
We split our cleaned data into an 80% training set and a 20% test set, using a standard **Logistic Regression** model.

6.1 Performance Results

Our model achieved an overall accuracy of **93%** on the test dataset.

- **Positive Class (1):** Precision of 0.93, Recall of 0.99, and F1-score of 0.96. The model does a great job identifying positive reviews because it had plenty of examples to learn from.
- **Negative Class (0):** Precision of 0.83, Recall of 0.49, and F1-score of 0.61. While the precision is decent (when the model flags a review as negative, it is correct 83% of the time), the recall is lower due to having fewer negative training samples overall.
- **Live Testing on Sample Sentences:** We tested the trained pipeline on custom reviews to check how it handles realistic feedback:
 - *"The fabric is very cheap, stitching came off, and the fit is terrible."* Predicted: **Negative (95.5% confidence)**.
 - *"Absolutely gorgeous dress! Soft material and fits true to size."* Predicted: **Positive (99.0% confidence)**.

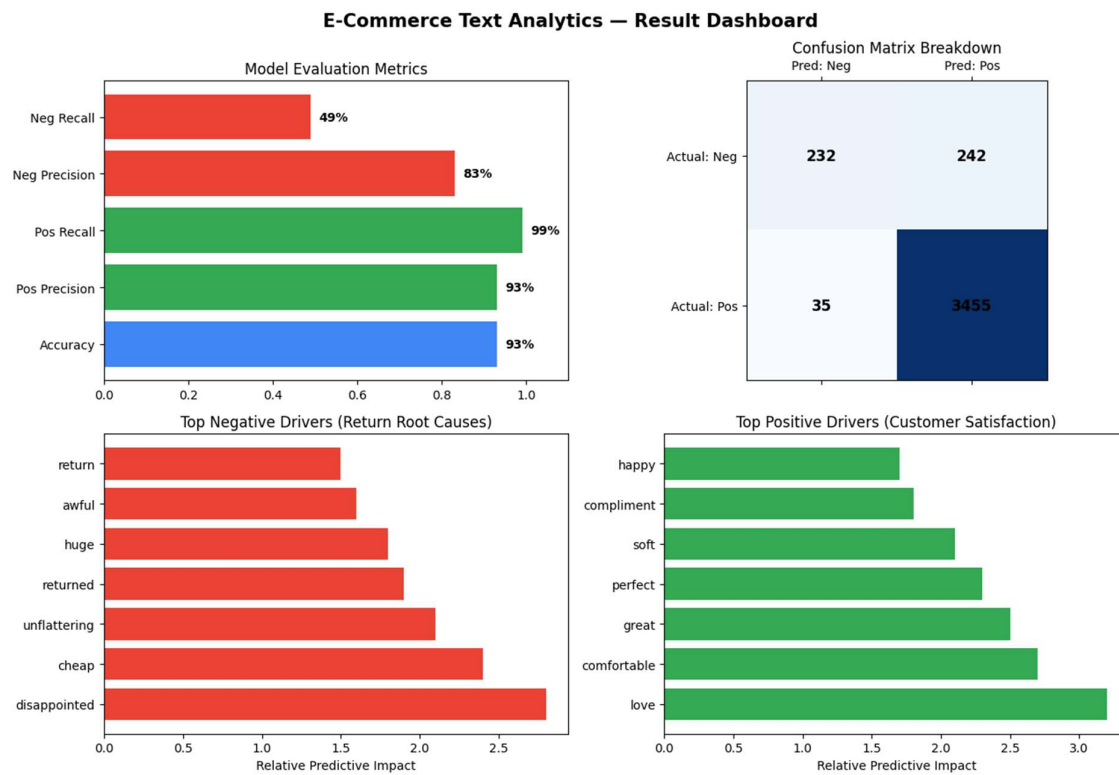
Classification Report:				
	precision	recall	f1-score	support
Negative (0)	0.83	0.49	0.61	474
Positive (1)	0.93	0.99	0.96	3490
accuracy			0.93	3964
macro avg	0.88	0.74	0.79	3964
weighted avg	0.92	0.93	0.92	3964



(Figure 5 — Test set classification metrics and confusion matrix results.)

6.2 Consolidated Result Dashboard

To make these results simple to present to business stakeholders, we combined our model accuracy, confusion matrix numbers, and top keyword drivers into a single dashboard visual.



(Figure 6 — Complete dashboard showing model metrics and top positive/negative keywords.)

7. Strategic Business Recommendations

Based on the keyword patterns and model findings, we recommend three concrete business actions:

1. **Improve Sizing Charts & Fit Advice:**

Negative reviews frequently complain about items being *huge*, *loose*, or *unflattering*. Sizing confusion is clearly the number one reason people return clothes. Adding precise measurements, model height references, and fabric stretch indicators to product pages will help customers pick the right size the first time and bring down return costs.

2. **Run Stricter Quality Checks on Fabric:**

Words like *cheap*, *thin*, and *disappointed* show that some garments look much better in website photos than in person. Setting strict quality standards for fabric thickness and stitching before shipping can help avoid bad reviews and customer disappointment.

3. **Set Up Automated Negative Review Alerts:**

This NLP pipeline can be hooked into the customer support portal. Whenever a review is predicted as negative with high confidence, the system can automatically create a ticket so the support team can offer a replacement or return assistance before the customer completely churns.

8. Project Limitations & Future Scope

Limitations

- **Imbalanced Data:** Because there were far fewer negative reviews than positive ones, our model struggled with negative recall (0.49), occasionally missing subtle complaints.
- **Word Context & Sarcasm:** TF-IDF looks at individual words and misses phrasing context, so it can misinterpret sarcasm or negations like "not bad at all".
- **Skipped Neutral Reviews:** We dropped 3-star reviews to keep the model simple, but neutral reviews often contain helpful mixed feedback.

Future Scope

- **Class Balancing:** We plan to experiment with balancing techniques like SMOTE or adjusting class weights to improve negative recall.
- **Advanced Transformer Models:** Moving to deep learning models like BERT or DistilBERT would help capture context and sarcasm much better than basic TF-IDF.
- **Aspect-Based Analysis:** Breaking down sentiment by specific product features (like evaluating fabric, fit, and delivery separately) would give more detailed insights for merchandising teams.

Live Deployment: The model could be packaged into a simple web app using Streamlit or FastAPI to analyse customer reviews in real time.



Bengaluru

NH 207, Nagadenehali,
Doddaballapur Taluk,
Bengaluru - 561203,
Karnataka, India

Hyderabad

Rudraram, Patancheru
mandal, Hyderabad -
502329, Telengana,
India

Visakhapatnam

Gandhi Nagar, Rushikonda,
Visakhapatnam- 530045,
Andhra Pradesh, India



Bengaluru



Visakhapatnam

GIMSR

GITAM Medical College Rd,
Gandhi Nagar, Rushikonda,
Visakhapatnam - 530045
Andhra Pradesh, India

Bengaluru City Office

No 5/1, First Floor, Prestige
Terraces, Union Street,
Infantry Road, Bengaluru -
560003 Karnataka, India

Hyderabad City Office

6th floor, TP House, Cyber
Hills Colony, VIP Hills,
Jaihind Enclave, Madhapur,
Hyderabad - 500081,
Telangana, India



Hyderabad



**GIMSR
Visakhapatnam**